

Assume the outputs from last layer are $\hat{y}$ then softmax activation

$$\hat{p}_i = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

$$L = - \sum_i y_i \log \hat{p}_i$$

$$\frac{\partial \hat{p}_i}{\partial \hat{y}_k} = \begin{cases} \hat{p}_k \cdot (1 - \hat{p}_k) & \text{if } i = k \\ -\hat{p}_k \hat{p}_i & \text{if } i \neq k \end{cases}$$

$$\frac{\partial L}{\partial \hat{y}_k} = - \sum_i y_i \frac{\partial \log \hat{p}_i}{\partial \hat{y}_k}$$

$$= - \sum_i y_i \frac{1}{\hat{p}_i} \frac{\partial \hat{p}_i}{\partial \hat{y}_k}$$

$$= - \sum_i y_i (1 - \hat{p}_i) + \sum_i y_i \hat{p}_i$$

$$= - \sum_{i=k} y_i (1 - \hat{p}_k) + \sum_{i \neq k} y_i \hat{p}_k$$

$$= -y_k + \sum_i y_i \hat{p}_k$$

$$= -y_k + \hat{p}_k \sum_i y_i$$

prob after softmax (pointing to $\hat{p}_k$)

$$\frac{\partial L}{\partial \hat{y}_k} = \hat{p}_k - y_k$$

ground-truth indicator (pointing to y_k)

output before softmax (pointing to $\hat{y}_k$)

Example

Use training example:

Input: $\vec{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ Target: $z = 3$

Weights: Initially all are 0.5.

Activation: ReLU.

Loss: Sum of squares error

$$L = \frac{1}{n} \sum \sum (z_k - \hat{z}_k)^2$$

$$L = \frac{1}{2} \sum \sum (z_k - \hat{z}_k)$$

Forward:

$$r_1 = v_{0,1}x_0 + v_{1,1}x_1 + v_{2,1}x_2 \quad y_1 = 2$$

$$= 2$$

$$r_2 = 2$$

$$y_2 = 2$$

$$\begin{aligned} s_1 &= w_0 y_0 + w_1 y_1 + w_2 y_2 \\ &= 2.5 \end{aligned}$$

$$\hat{z}_1 = 2.5$$

Loss:

$$L = \frac{1}{2} \sum \sum (z_k - \hat{z}_k)^2$$

$$= \frac{1}{2} (z_1 - \hat{z}_1)^2$$

$$= \frac{1}{8} = 0.125$$

Backpropagation:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial s_k} \frac{\partial s_k}{\partial w_j}$$

$$\frac{\partial L}{\partial s_k} = \frac{\partial L}{\partial \hat{z}_k} a'(s_k)$$

$$\frac{\partial s_k}{\partial w_0} = 1$$

$$\frac{\partial L}{\partial s_k} = \frac{\partial L}{\partial \hat{z}_k} \cdot (-1)$$

$$= (\hat{z}_k - z_k) \cdot a'(s_k)$$

$$= (2.5 - 3) \cdot 1$$

$$= -0.5$$

$$\frac{\partial L}{\partial w_0} = 1$$

$$\frac{\partial s_k}{\partial w_1} = 2$$

$$\frac{\partial s_k}{\partial w_2} = 2$$

$$\frac{\partial L}{\partial w_0} = -0.5, \quad \frac{\partial L}{\partial w_1} = -1, \quad \frac{\partial L}{\partial w_2} = -1$$

$$\nabla L(\vec{w}) = [-0.5, -1, -1]$$

$$\frac{\partial L}{\partial v_{i,j}} = \frac{\partial L}{\partial r_j} \frac{\partial r_j}{\partial v_{i,j}}$$

$$\frac{\partial L}{\partial r_j} = \sum_k \frac{\partial L}{\partial s_k} w_j \cdot a'(r_j)$$

$$= -0.5 \cdot w_j \cdot a'(r_j)$$

$$\frac{\partial L}{\partial r_1} = -0.5 \cdot 0.5 \cdot 1 = -0.25$$

$$\frac{\partial L}{\partial r_2} = -0.5 \cdot 0.5 \cdot 1 = -0.25$$

$$\frac{\partial L}{\partial v_{0,1}} = \frac{\partial L}{\partial r_1} \cdot x_0 = -0.25 \quad \frac{\partial L}{\partial v_{0,2}} = -0.25$$

$$\frac{\partial L}{\partial v_{1,1}} = \frac{\partial L}{\partial r_1} \cdot x_1 = -0.25 \quad \frac{\partial L}{\partial v_{1,2}} = -0.25$$

$$\frac{\partial L}{\partial v_{2,1}} = \frac{\partial L}{\partial r_1} \cdot x_2 = -0.5 \quad \frac{\partial L}{\partial v_{2,2}} = -0.5$$

$$\nabla L(\vec{r}) = [-0.25, -0.25]$$

Update with gradient descent: $\alpha = 0.1$

$$w_0 \leftarrow w_0 - \alpha \cdot \frac{\partial L}{\partial w_0}$$

$$\leftarrow 0.5 - 0.1 \cdot (-0.5)$$

$$\leftarrow 0.55$$

$$w_1 \leftarrow 0.5 - 0.1 \cdot (-1)$$

$$\leftarrow 0.6$$

$$w_2 \leftarrow 0.6$$

$$v_{0,1} \leftarrow v_{0,1} - \alpha \cdot \frac{\partial L}{\partial v_{0,1}}$$

$$\leftarrow 0.5 - 0.1 \cdot (-0.25)$$

$$\leftarrow 0.525$$

$$v_{1,1} \leftarrow 0.525$$

$$v_{2,1} \leftarrow 0.55$$

$$v_{0,2} \leftarrow 0.525$$

$$v_{1,2} \leftarrow 0.525$$

$$v_{2,2} \leftarrow 0.55$$

Forward:

$$r_1 = v_{0,1} \cdot x_0 + v_{1,1} \cdot x_1 + v_{2,1} \cdot x_2 = 2.15$$

$$y_1 = 2.15$$

$$r_2 = v_{0,2} \cdot x_0 + v_{1,2} \cdot x_1 + v_{2,2} \cdot x_2 = 2.15$$

$$y_2 = 2.15$$

$$s_1 = w_0 \cdot y_0 + w_1 \cdot y_1 + w_2 \cdot y_2 = 3.13$$

$$\hat{z}_1 = 3.13$$

Loss:

$$L = \frac{1}{2} \sum \sum (z_k - \hat{z}_k)^2$$

$$= \frac{1}{2} (3 - 3.13)^2$$

$$= 0.00845$$

Unsupervised Learning in Neural Networks

Self-Organizing Maps (Kohonen Networks)

Consider neurons in the map to be prototypes

1. Given input x , compute most similar neuron A .
2. Adjust A to make it more similar to x .
3. Adjust neighbours of A to be more similar to x (to some lesser extent).
4. Repeat.